

EECE 4830-5830 Network Design
Dr. Vinod Vokkarane

Project Phase 2: Implement RDT 2.2 over an unreliable UDP channel with bit-errors

Project description: The TCP/IP stack has five layers, namely application, transport, network, link, and physical. In Phase 1, each student implemented the standard user datagram protocol (UDP) sockets. The intention was to transfer a file (say JPEG) between a UDP client process and a UDP server process. Also, each of you implemented reliable data transfer service over the reliable UDP connection developed. In Phase 2, each of you have to implement the RDT 2.2 protocol described in Section 3.4.1, page 218-227 of the course textbook. The finite state machines (FSM) of the sender and received are given below:

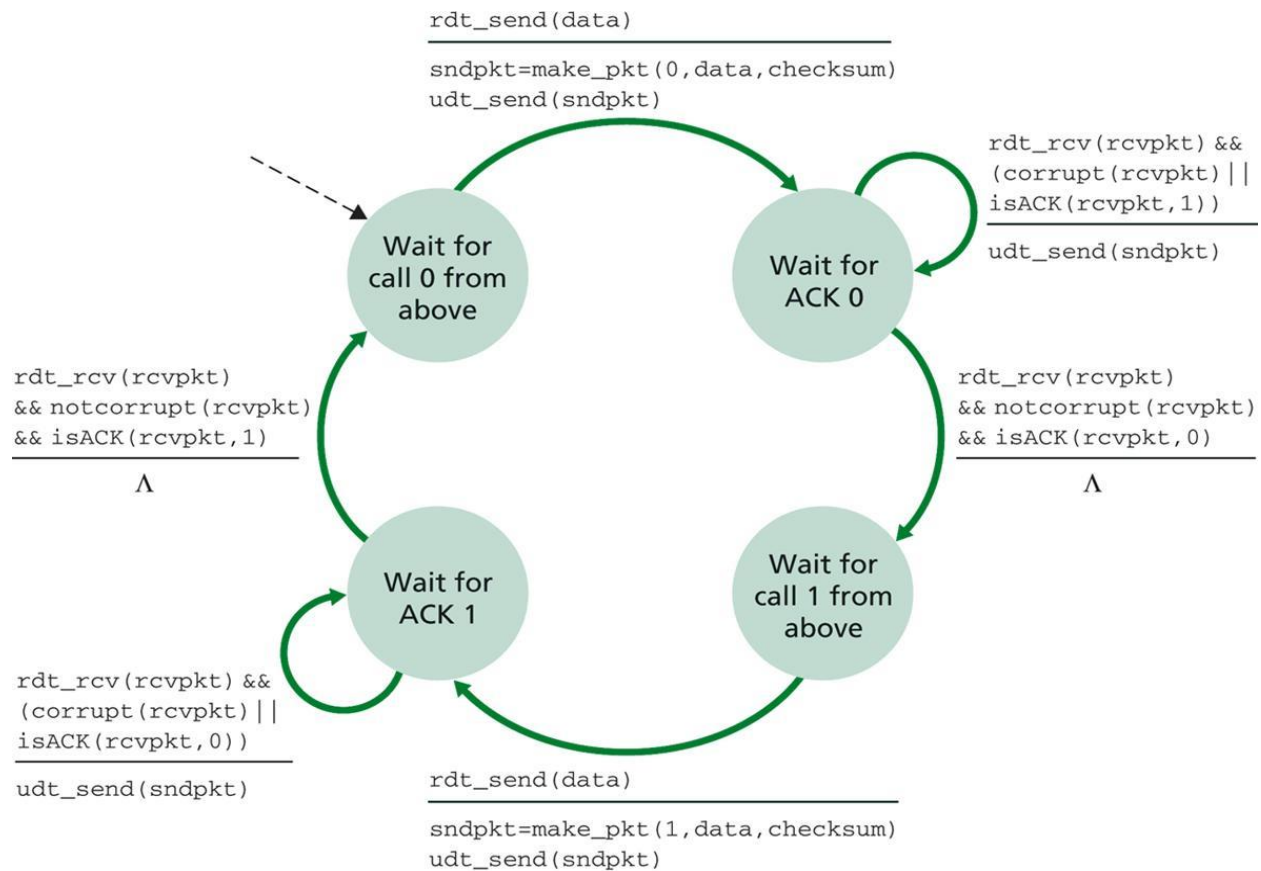


Figure 3.13 ♦ rdt2.2 sender

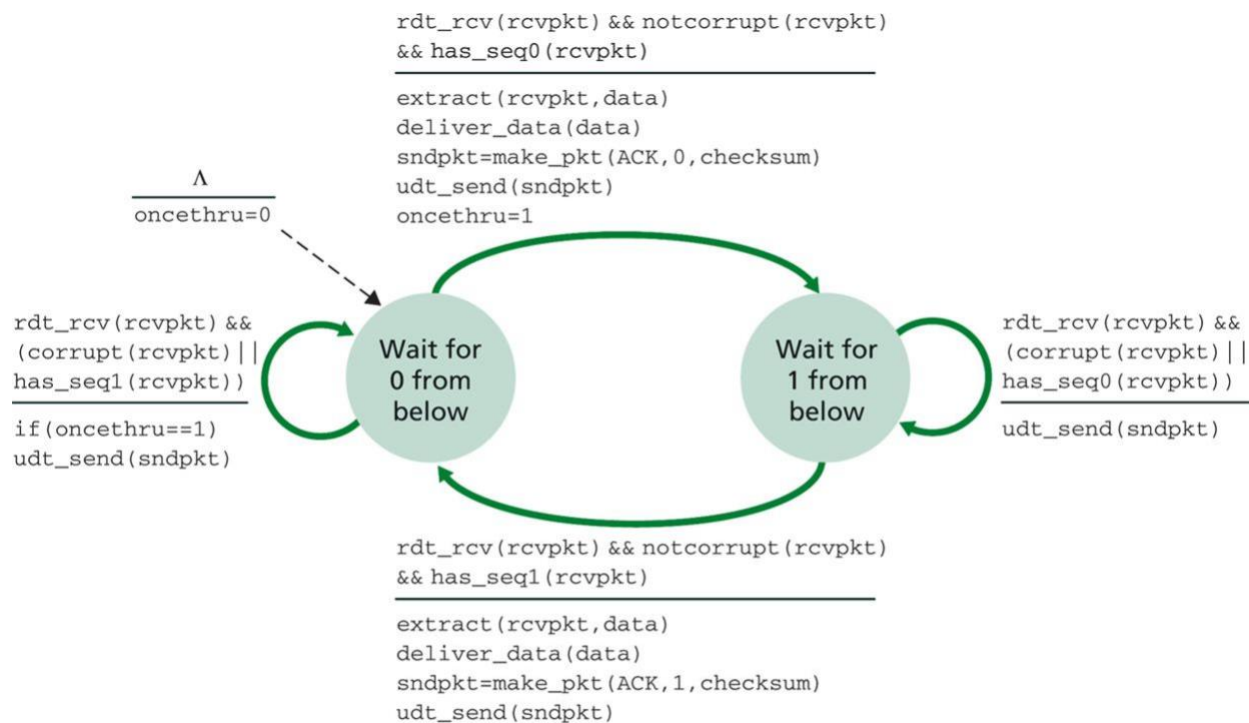


Figure 3.14 ♦ rdt2.2 receiver

The following are the basic implementation steps:

1. Pick a transfer file - BMP image file, easier to identify loss of packets in an image file (lost pixels).
2. Make_packet function - parses the image file and breaks it down to several packets (set a fixed packet size, say 1024 bytes).
3. Use the UDP sockets (Phase 1) to send and receive RDT2.2 packets (UDP sockets are unreliable, so they simulate the unreliable Internet IP (network) layer).
4. Implement: Sequence Numbers (to identify duplicates), Checksum, and ACKs (remember, RDT 2.2 is a NAK-free protocol).
5. The RDT2.2 receiver should assemble packets in order and deliver the entire transfer file to the application.

In this phase, you will need to implement ALL the following data transfer scenarios:

1. **Option 1 - No loss/bit-errors.**
2. **Option 2 - ACK packet bit-error:** you need to intentionally change the data bits of the received ACK packet at the sender and implement suitable recovery mechanism.
3. **Option 3 - Data packet bit-error:** you need to intentionally change the data bits of the received DATA packet at the receiver and implement suitable recovery mechanism.

Implementation: Team effort

Expectations: In this phase of the project, you will learn about the basic principles used to provide non-pipelined reliable data transfer over a data channel with bit errors.

Programming language: Python.

Deliverables:

1. **DESIGN_DOCUMENT.pdf/md**
 1. If submitting on BlackBoard, it MUST be a PDF
 2. If you only want to submit your GitHub repo, it can remain a MD
2. **README.pdf/md**
3. **Plot charts** for completion time for all your **3 options** for transferring the same file at **0% loss/error to 95% loss/error in increments of 5%.**
 1. Note:
 1. Each point on the plot MUST be an average of multiple runs (5) at a given loss/error rate (make sure your file isn't massive, you'll be running for days ☺).
 2. For accuracy remove print/debug statements to the output console statements while measuring the completion time for the chart.
4. **Individual Contribution (contribution.txt/doc):**

This is confidential submission for each member

 1. Teammate rating between 1 (not good) – 5 (excellent) on
 1. Project time commitment

2. Design contribution
3. Coding contribution
4. Debugging contribution
5. Report preparation

All submission will be inspected using the [Plagiarism detection software](#).

References:

1. Socket Programming
 - Python: Socket Programming (Textbook Sec 2.7)
2. UDP: [RFC768](#)
3. Principles of Reliable Data Transfer, Section 3.4.1, Kurose/Ross (7th Ed).